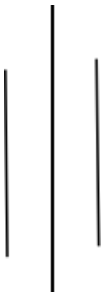


Tribhuvan University

Faculty of Humanities and Social Science

Swastik college, Bhaktapur



Cloud Computing

Lab Report

Submitted By:

Swosti Makaju(113102097)

BCA 7th Semester

Submitted To:

Sujan Poudel

Lecturer, BCA Department

Swastik college, Bhaktapur

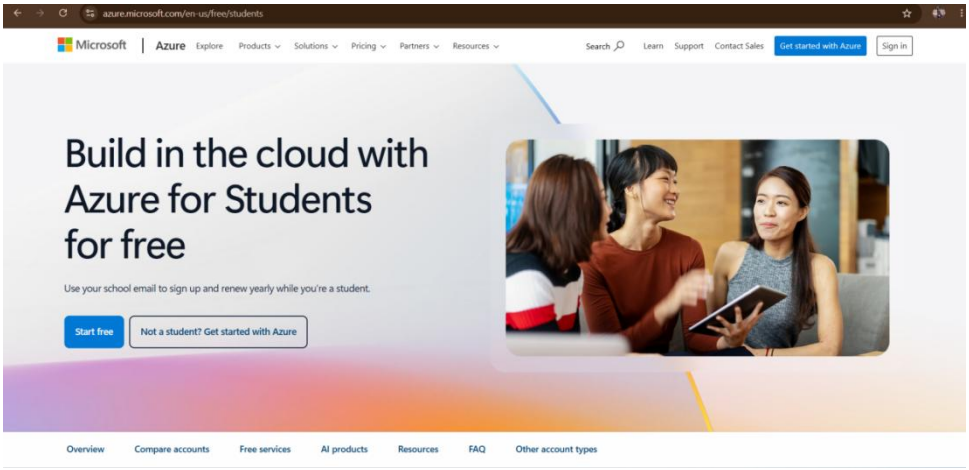
Lab 1: Getting Started with Azure

Objective: Understand the Azure portal, subscription model, and cloud terminology.

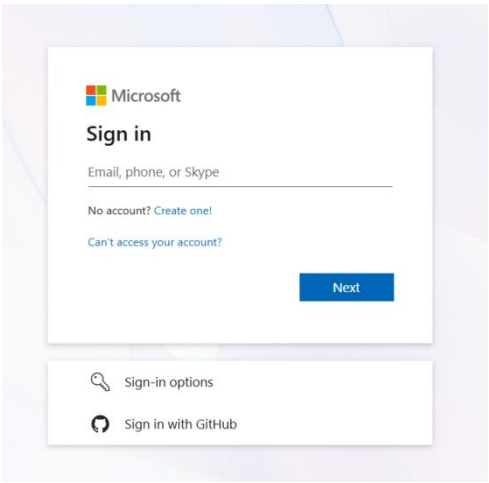
Tasks:

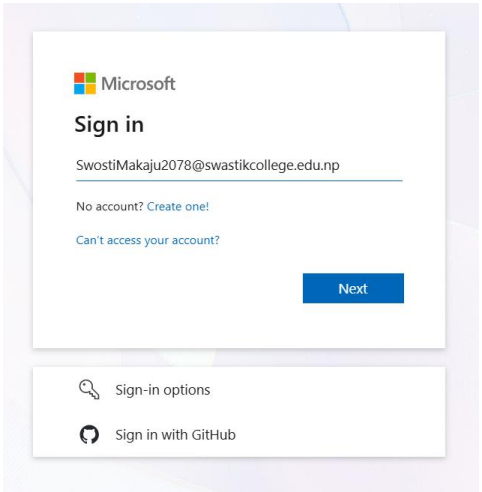
1. Create an Azure Account

- Visit: https://azure.microsoft.com/en-us/free/students



- Click **Start Free** and sign in with a Microsoft account (use a school/university email).



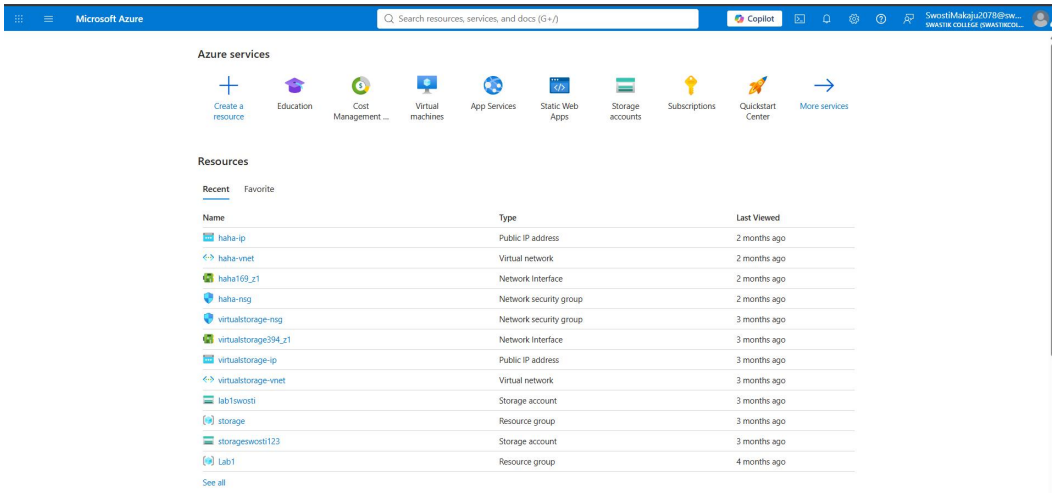


- Select **Student Option** to get $100 credits via Azure for Students.
- Complete the sign up and verification (phone number + identity verification).

Note: No credit card is needed for student accounts.

2. Explore Azure Portal and Cloud Shell

- Login to the Azure Portal.



- Familiarize with **Dashboard, Resource Groups, All Services, Marketplace**

3. Create a Resource Group

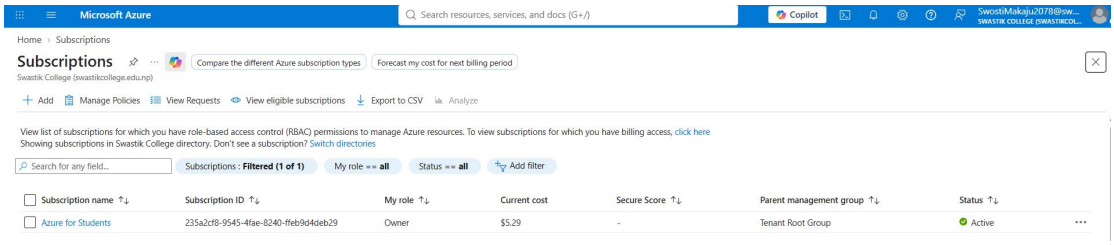
A resource group is a logical container for your Azure resources.

- Via Portal:

- Go to **Resource Groups → Create**
- Resource Group Name: lab1swosti
- Region: South India (or nearest region)

4.Verify Azure Subscription

- Go to Subscriptions and verified the Azure for students subscription.
- Confirmed that the subscription status was Active.

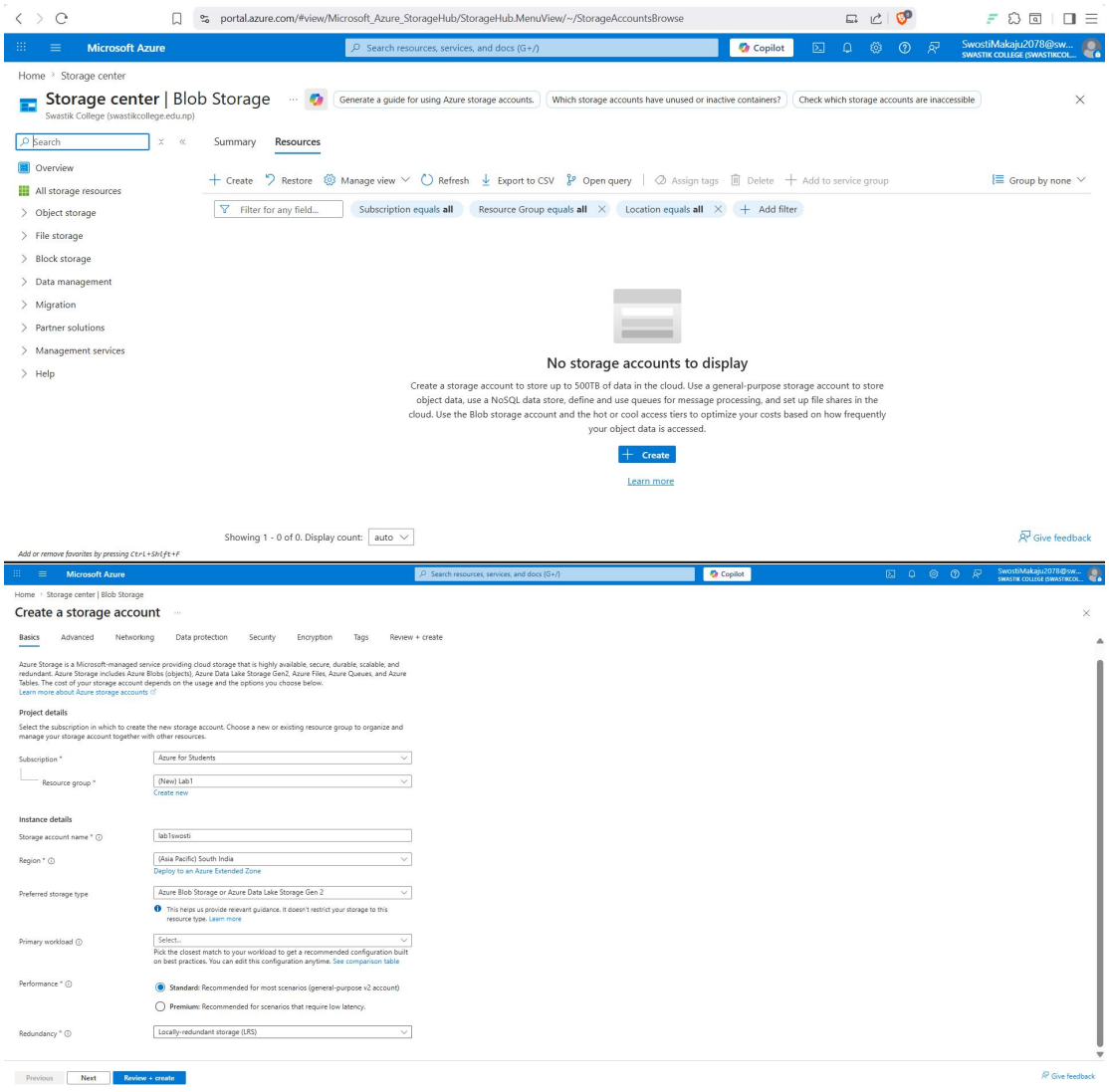


Lab 2: Azure Storage Services

Objective:
Learn to create and manage Azure Blob Storage, including uploading files and accessing them via public URLs using both the Portal and Azure CLI.

Via Portal:

1. Go to **Storage accounts → Create**

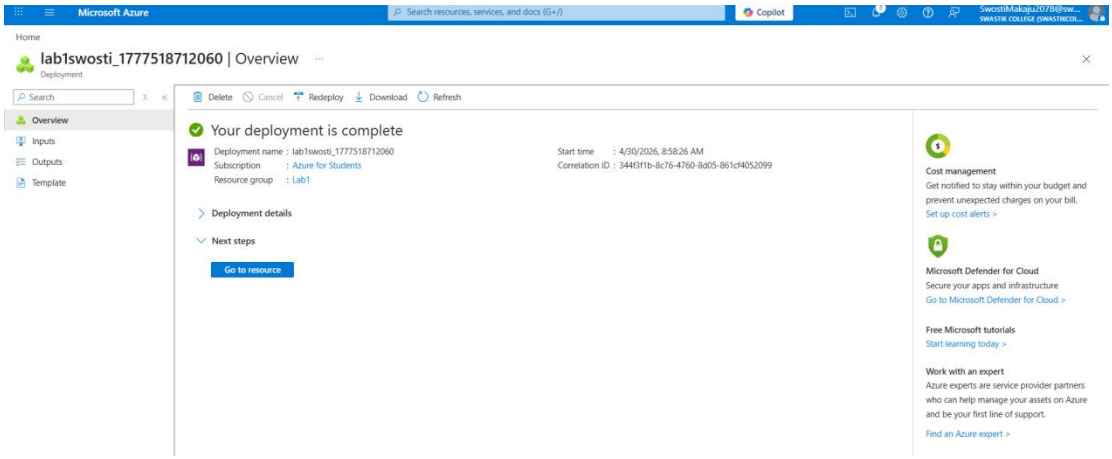


2. Fill in the details:

- Subscription: (Select your active subscription)
- Resource Group: Lab1 (or create a new one)
- Storage Account Name: lab1swosti

- Region:South India
- Performance: Standard
- Redundancy: Locally-redundant storage (LRS)

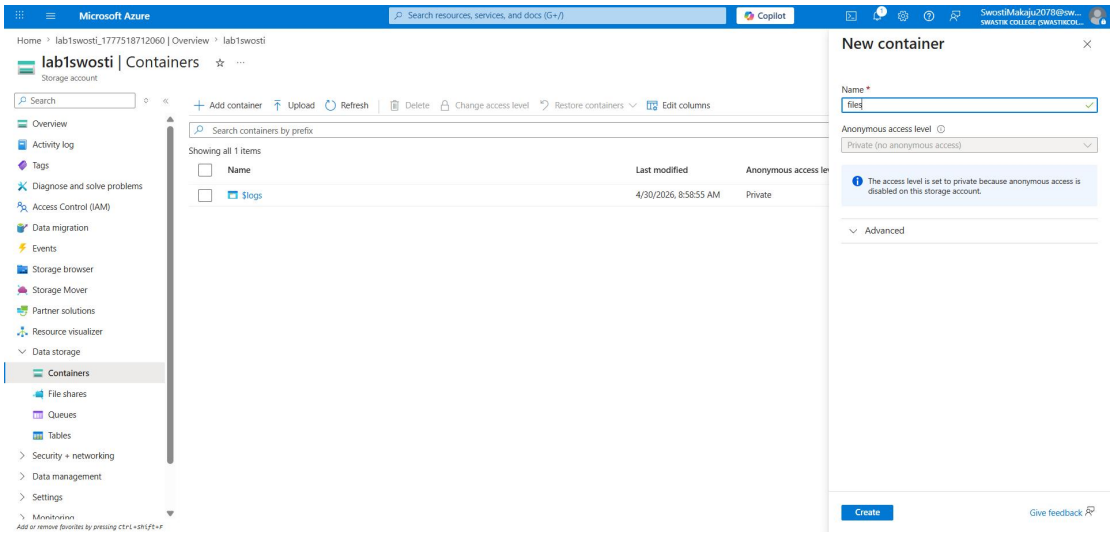
3. Click **Review + Create**, then **Create**



4. Create a Blob Container

Via Portal:

- Navigate to your newly created Storage Account
- Under **Data storage**, select **Containers**
- Click **+ Container**



Lab 3: Azure Virtual Machines

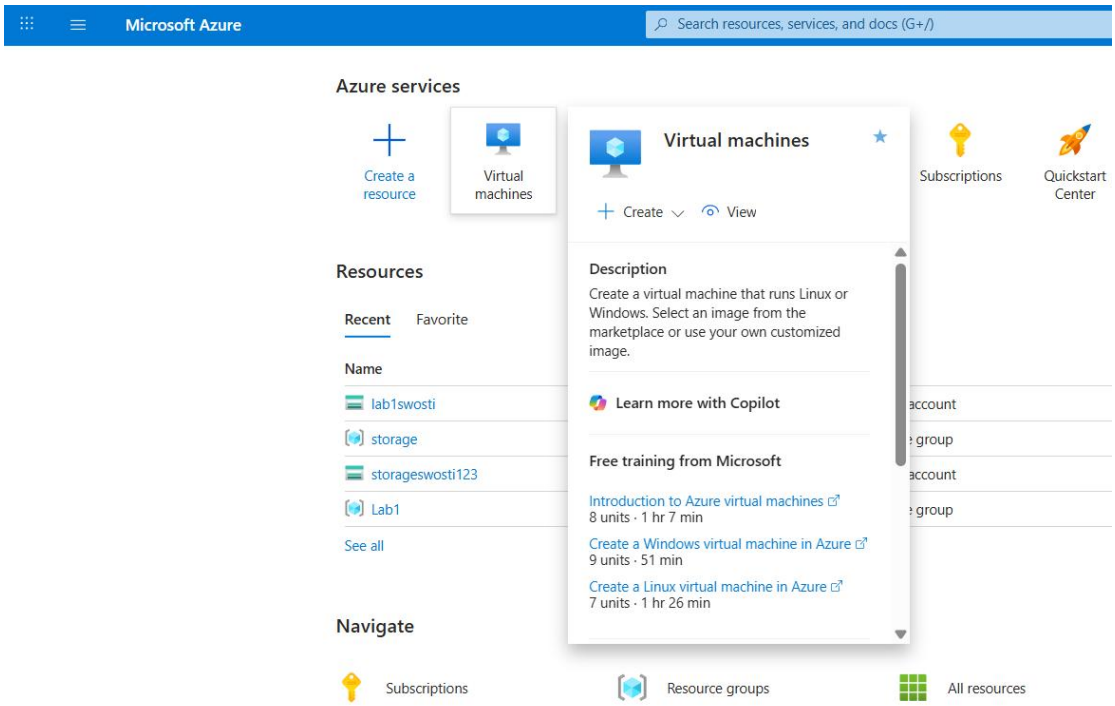
Objective: Deploy a virtual machine and perform basic server tasks.

Deploy a Linux virtual machine on Azure, connect to it via SSH, and host a static website using Apache or Nginx.

1. Create a Linux-based VM

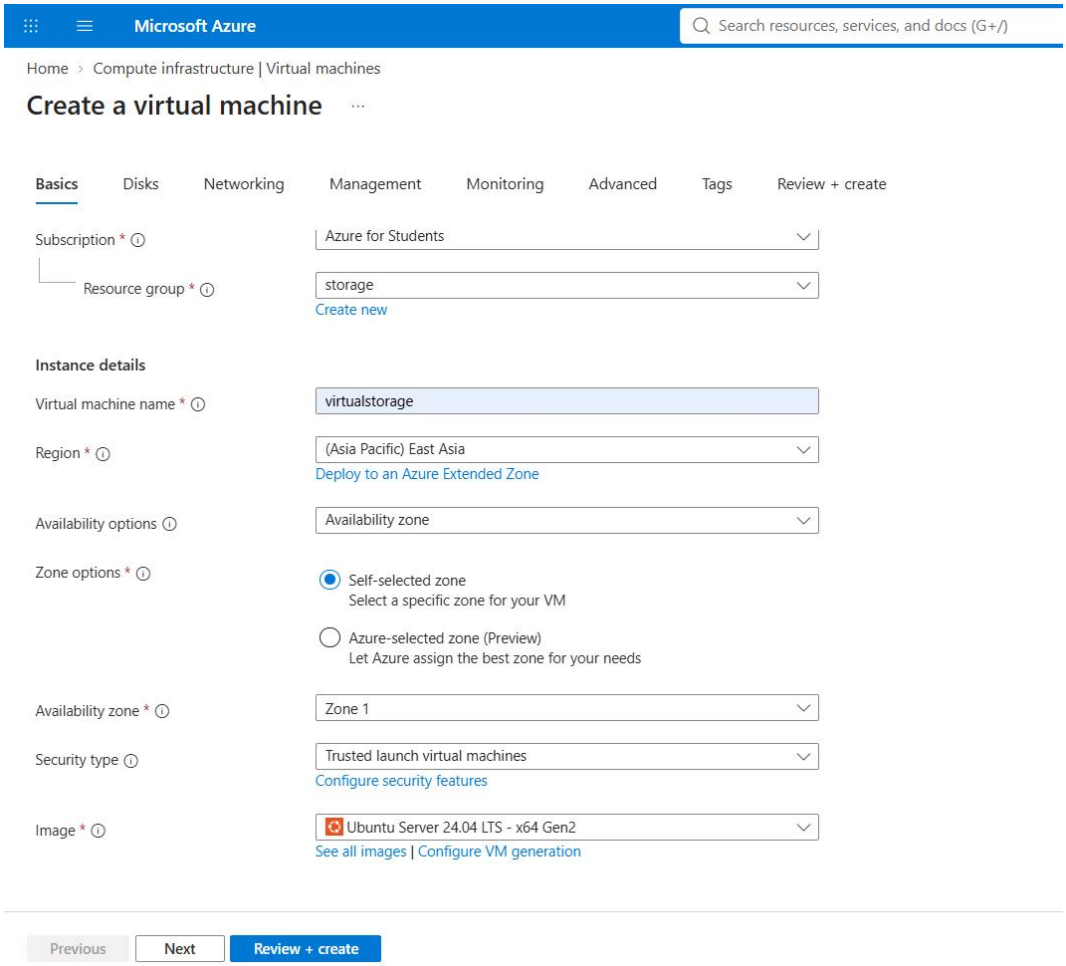
Via Portal:

1. Go to Virtual Machines → Create

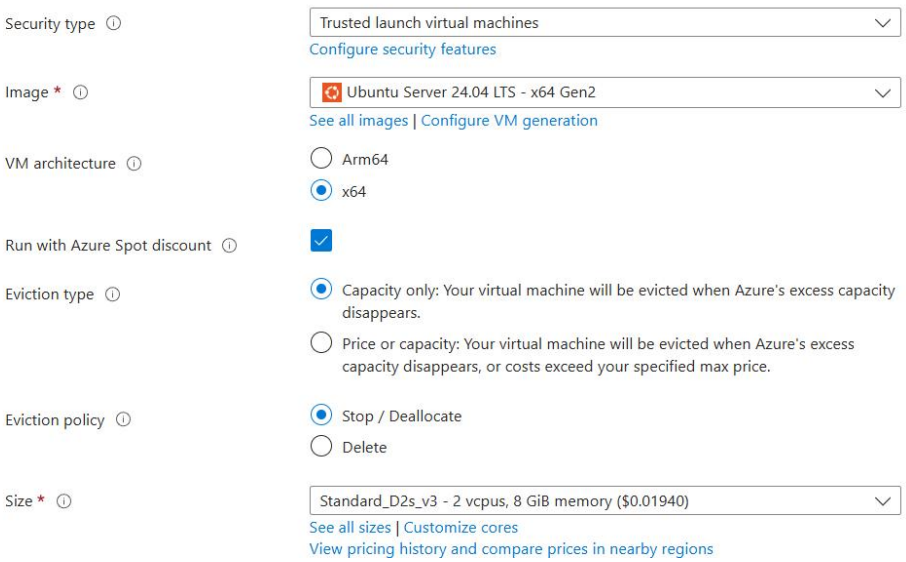


2. **Fill in the following:**

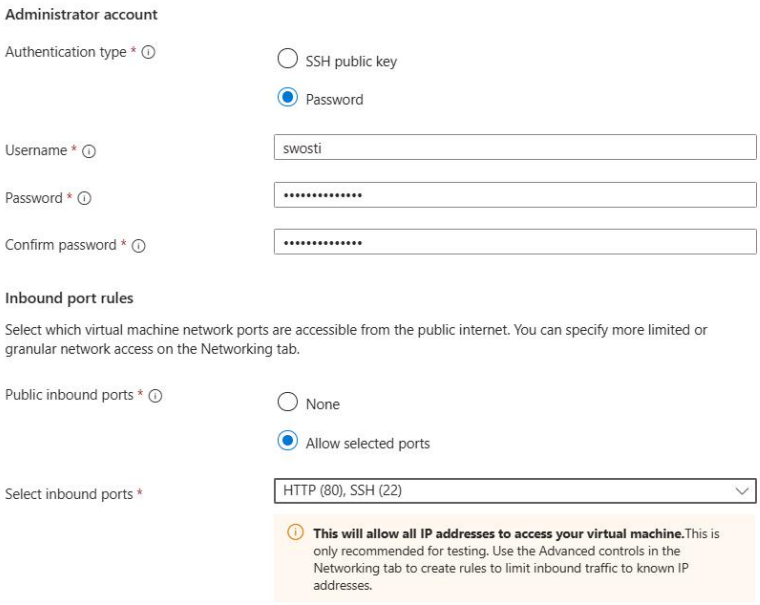
- Subscription: Your active one
- Resource Group: storage
- VM name: virtualstorage
- Region: East US (or nearby)



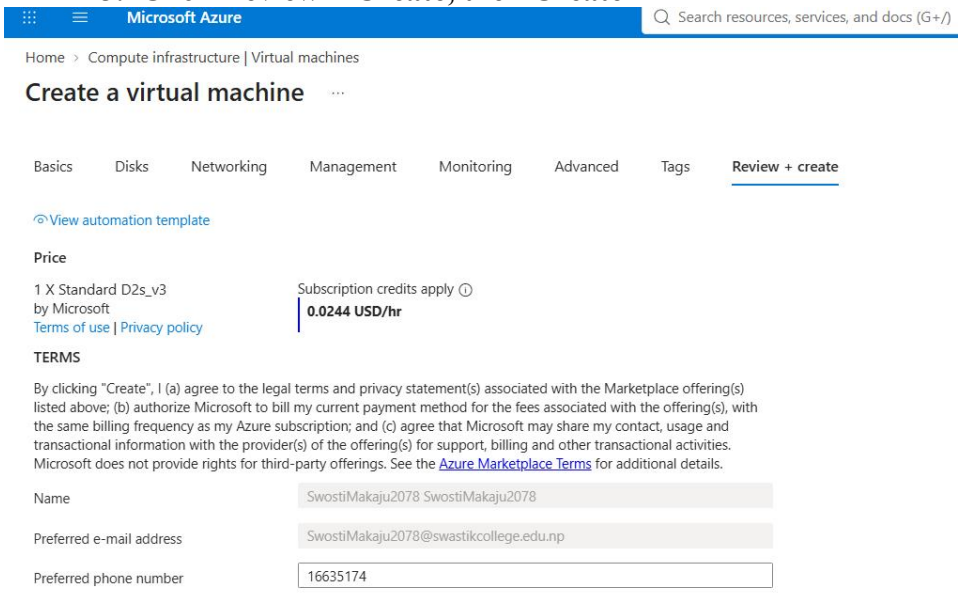
- Image: Ubuntu Server 20.04 LTS
- Size: Standard B1s (Free tier if available)



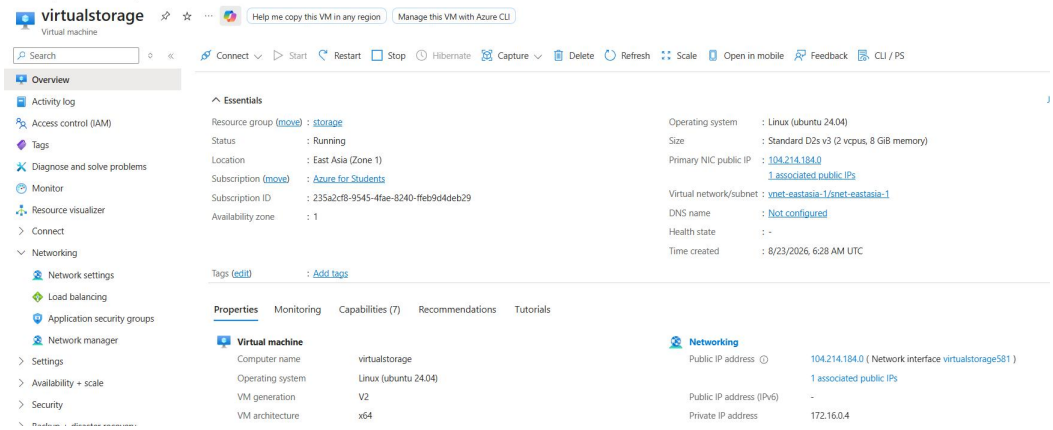
- Authentication: SSH (generate or upload key)
- Allow inbound ports: SSH (22), HTTP (80)



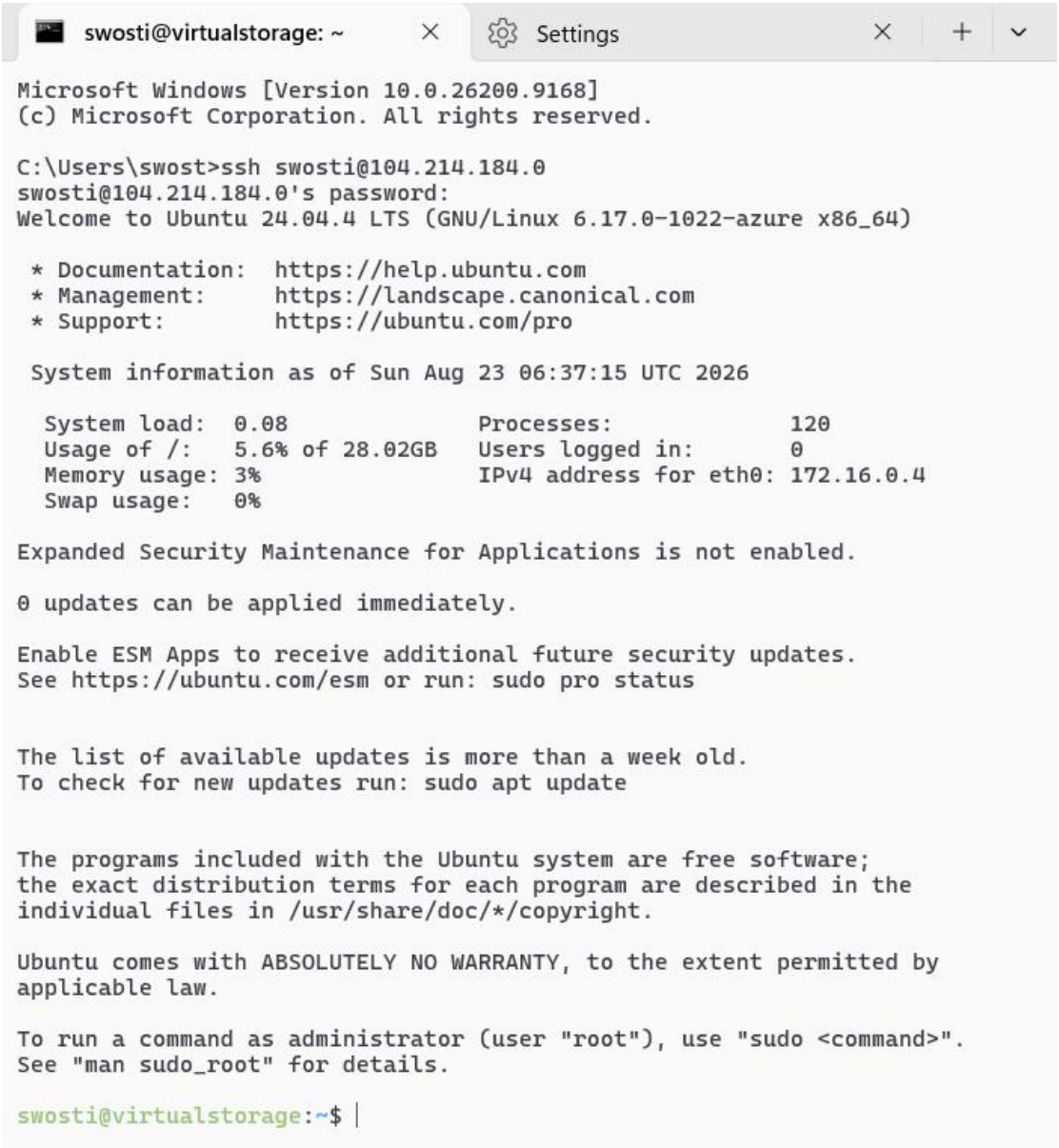
3. Click Review + Create, then Create



4. After deployment, go to the VM → Note down the Public IP address then host a static website



5. Host a Static Website



Lab 4 NGINX Configuration

1. Do SSH connection with your virtual machine

Here the virtual machine name is “newvm” and the ip is “4.240.110.169” use your configuration. After doing ssh enter password.

ssh <swosti>@<4.240.110.169>

ssh swosti@4.240.110.169
Swosti12345678

```
C:\Users\swost>ssh swosti@4.240.110.169
The authenticity of host '4.240.110.169 (4.240.110.169)' can't be established.
ED25519 key fingerprint is SHA256:6xZhTJ+uqHLRnTXtFul1tAlz/FotfttCFhkm+KcmLc4.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '4.240.110.169' (ED25519) to the list of known hosts.
swosti@4.240.110.169's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1022-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Tue Aug 25 05:35:42 UTC 2026

  System load:  0.08              Processes:             121
  Usage of /:   5.6% of 28.02GB   Users logged in:       0
  Memory usage: 3%                IPv4 address for eth0: 172.16.0.4
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

swosti@newvm:~$
```

1. Install NGINX

Update system packages and install NGINX:

sudo apt update

```
swosti@newvm:~$ sudo apt update
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:6 http://azure.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:8 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:9 http://azure.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:10 http://azure.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
```

sudo apt install nginx -y

```
swosti@newvm:~$ sudo apt install nginx -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  nginx-common
Suggested packages:
  fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  nginx nginx-common
0 upgraded, 2 newly installed, 0 to remove and 35 not upgraded.
Need to get 568 kB of archives.
After this operation, 1602 kB of additional disk space will be used.
```

Verify that NGINX is running:

sudo systemctl status nginx

```
swosti@newvm:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-08-25 05:38:34 UTC; 50s ago
       Docs: man:nginx(8)
    Process: 2251 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 2253 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 2282 (nginx)
      Tasks: 3 (limit: 9433)
     Memory: 2.4M (peak: 5.3M)
        CPU: 22ms
     CGroup: /system.slice/nginx.service
             ├─2282 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─2285 "nginx: worker process"
             └─2286 "nginx: worker process"

Aug 25 05:38:34 newvm systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server.
Aug 25 05:38:34 newvm systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
```

2. Connect GitHub to the Virtual Machine

If this is the first time connecting GitHub:

1. Generate SSH key: ***ssh-keygen -t ed25519 -C"your_email@example.com"***

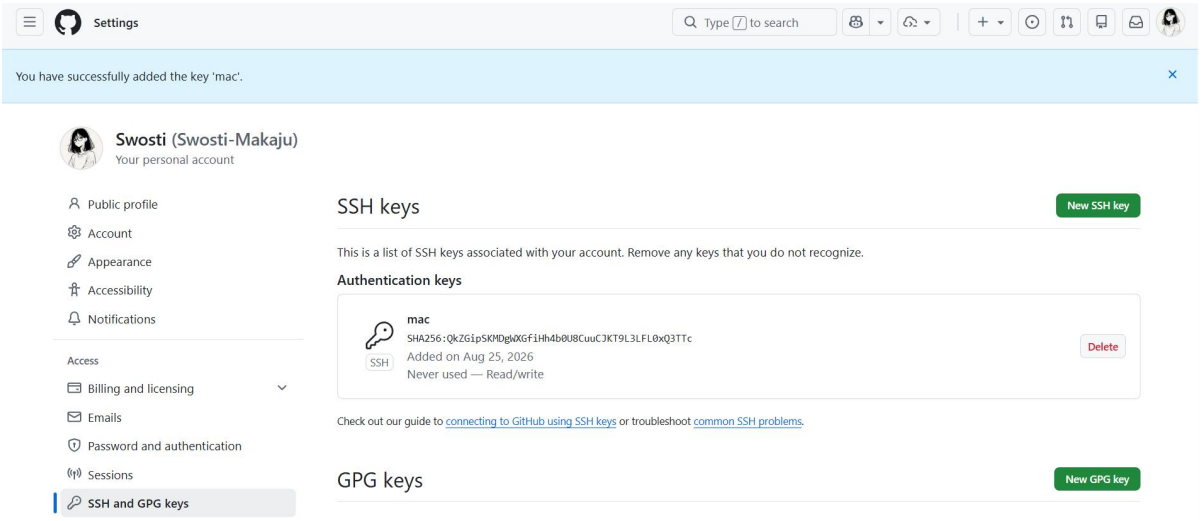
```
swosti@newvm:~$ ssh-keygen -t ed25519 -C "swosti.makaju@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/swosti/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/swosti/.ssh/id_ed25519
Your public key has been saved in /home/swosti/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:QkZGipSKMDgWXGfiHh4b0U8CuuCJKT9L3LFL0xQ3TTc swosti.makaju@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
|o.+*o++   . E    |
|=++.=* . o . .   |
|==.=. B o .      |
|*.= =o + .       |
|++ +. o S        |
|.o . = .         |
|  = = .          |
| . + o           |
|  . .            |
+----[SHA256]-----+
swosti@newvm:~$
```

2. View and copy your public key:

cat ~/.ssh/id_ed25519.pub

```
swosti@newvm:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIIkI5Mqynxar30KKeNsYDHXW6qHWLch1PsDHtRdhNq0l swosti.makaju@gmail.com
swosti@newvm:~$
```

3. Add it to GitHub → **Settings → SSH and GPG Keys → New SSH Key**.



Test the connection:

ssh -T git@github.com

```
swosti@newvm:~$ ssh -T git@github.com
The authenticity of host 'github.com (20.207.73.82)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
Hi Swosti-Makaju! You've successfully authenticated, but GitHub does not provide shell access.
```

3. Pull the Repository You Want to Host

Navigate to the directory where you want the project:

cd /var/www/

```
swosti@newvm:~$ cd /var/www
swosti@newvm:/var/www$
```

Clone the repository:

sudo git clone git@github.com:username/repository.git

If you get permission error then do;
sudo chown -R swosti:swosti /var/www

```
swosti@newvm:/var/www$ sudo chown -R swosti:swosti /var/www
```

```
swosti@newvm:/var/www$ git clone https://github.com/Swosti-Makaju/Cloud_Computing.git
Cloning into 'Cloud_Computing'...
remote: Enumerating objects: 17, done.
remote: Counting objects: 100% (17/17), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 17 (delta 4), reused 15 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (17/17), 5.50 KiB | 5.50 MiB/s, done.
Resolving deltas: 100% (4/4), done.
```

4. Run the Project & Install Dependencies

Go into the project folder:

cd repository

```
swosti@newvm:/var/www$ cd Cloud_Computing
swosti@newvm:/var/www/Cloud_Computing$ ls
README.md  app.py  requirements.txt  templates
```

Install dependencies (examples):

Python (FastAPI / Django / Flask)

sudo apt update && sudo apt install -y python3-venv python3-full
pip install -r requirements.txt

```
swosti@newvm:/var/www/Cloud_Computing$ sudo apt update && sudo apt install -y python3-venv python3-full
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
33 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
swosti@newvm:/var/www/Cloud_Computing$ python3 -m venv venv
swosti@newvm:/var/www/Cloud_Computing$ source venv/bin/activate
(venv) swosti@newvm:/var/www/Cloud_Computing$ pip install -r requirements.txt
Collecting flask (from -r requirements.txt (line 1))
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting azure-storage-blob (from -r requirements.txt (line 2))
  Downloading azure_storage_blob-12.30.0-py3-none-any.whl.metadata (26 kB)
Collecting python-dotenv (from -r requirements.txt (line 3))
  Downloading python_dotenv-1.2.3-py3-none-any.whl.metadata (29 kB)
Collecting blinker>=1.9.0 (from flask->-r requirements.txt (line 1))
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask->-r requirements.txt (line 1))
  Downloading click-8.4.2-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask->-r requirements.txt (line 1))
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask->-r requirements.txt (line 1))
```

6. Create an NGINX Site Configuration File

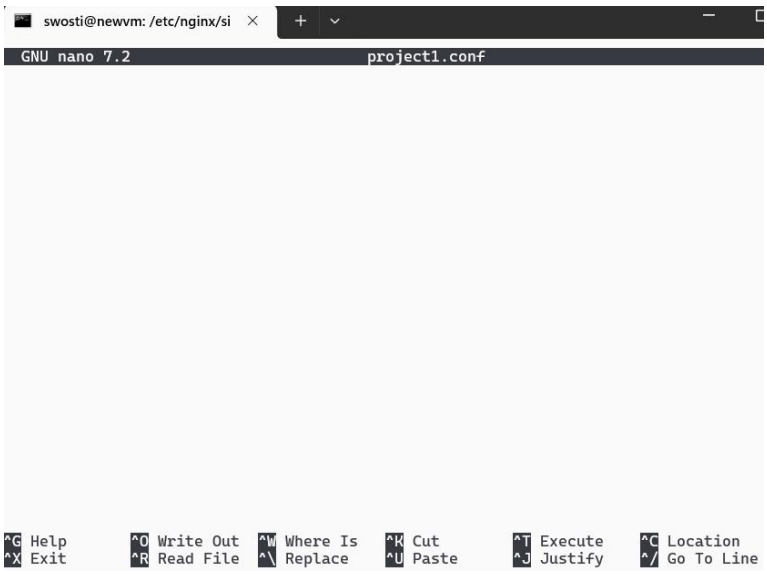
Go to NGINX's site-available directory:

cd /etc/nginx/sites-available/

```
(venv) swosti@newvm:/var/www/Cloud_Computing$ cd /etc/nginx/sites-available/
(venv) swosti@newvm:/etc/nginx/sites-available$ ls
default
```

Create a config file for your project:

sudo nano project1.conf



Example NGINX Config (Reverse Proxy)

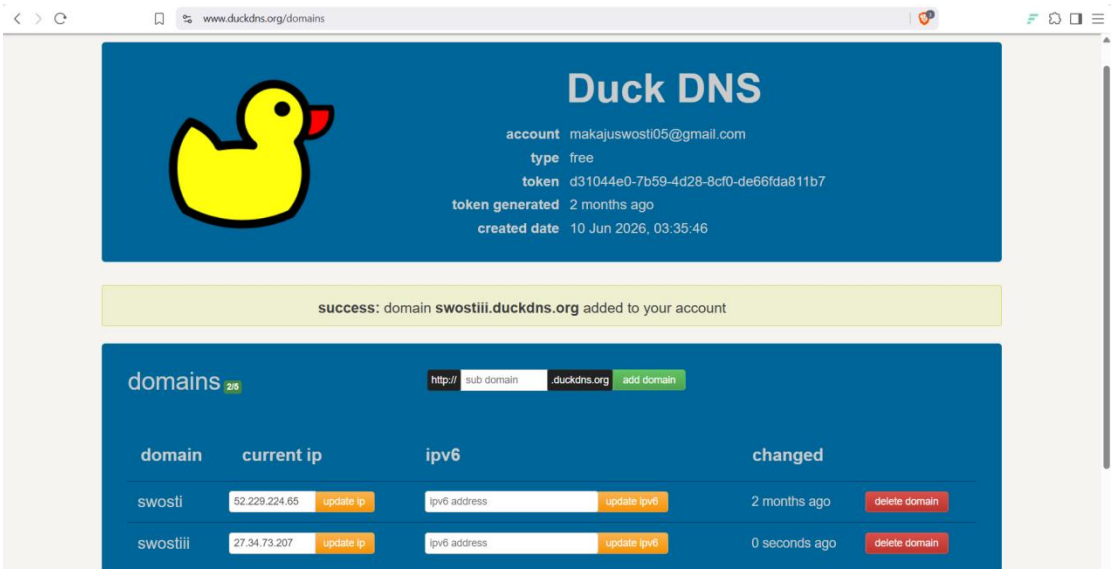
(Adjust port and folder paths as needed)

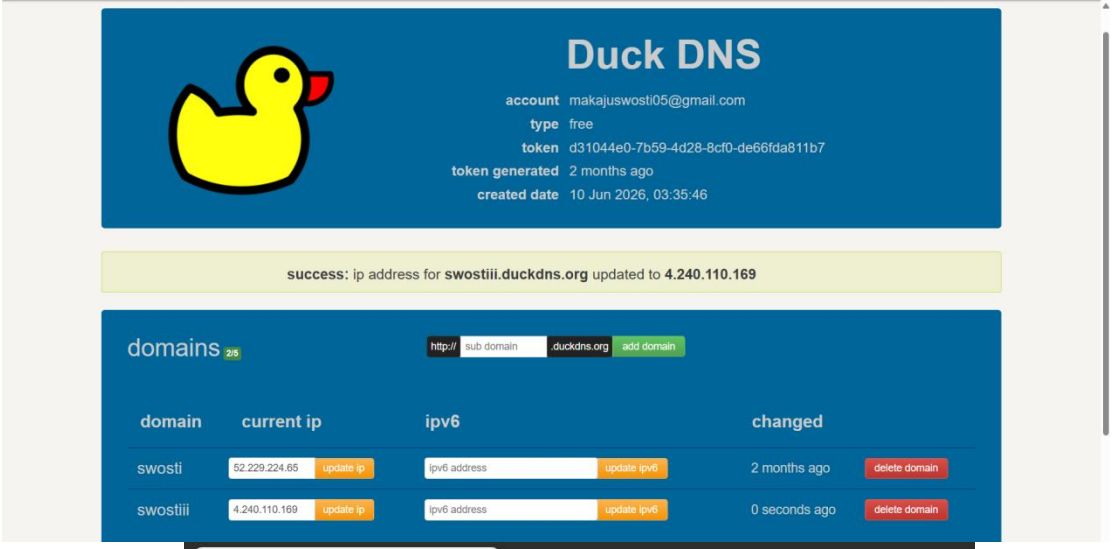
```
server {
    listen 80;
    server_name your_domain.com;

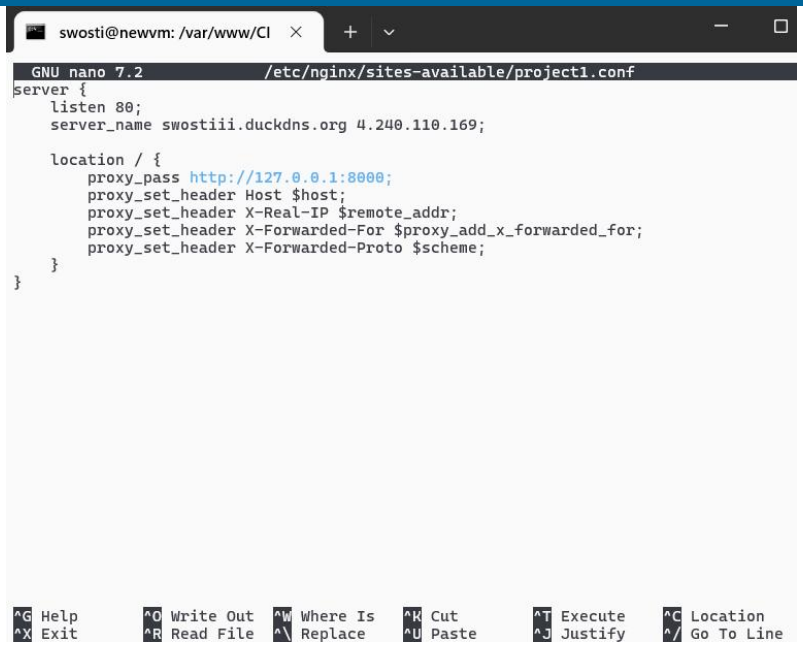
    location / {
        proxy_pass http://127.0.0.1:8000;   # Your app port
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Example

We need a domain name for this purpose so we are using the free [duckdns.org domain](duckdns.org) for the lab.


www.duckdns.org/domains
Duck DNS
account makajuswosti05@gmail.com
type free
token d31044e0-7b59-4d28-8cf0-de66fda811b7
token generated 2 months ago
created date 10 Jun 2026, 03:35:46
success: domain swostiii.duckdns.org added to your account
domains 2/5
http:// sub domain .duckdns.org add domain
domain current ip ipv6 changed
swosti 52.229.224.65 update ip ipv6 address update ipv6 2 months ago delete domain
swostiii 27.34.73.207 update ip ipv6 address update ipv6 0 seconds ago delete domain


Duck DNS
account makajuswosti05@gmail.com
type free
token d31044e0-7b59-4d28-8cf0-de66fda811b7
token generated 2 months ago
created date 10 Jun 2026, 03:35:46
success: ip address for swostiii.duckdns.org updated to 4.240.110.169
domains 2/5
http:// sub domain .duckdns.org add domain
domain current ip ipv6 changed
swosti 52.229.224.65 update ip ipv6 address update ipv6 2 months ago delete domain
swostiii 4.240.110.169 update ip ipv6 address update ipv6 0 seconds ago delete domain


swosti@newvm: /var/www/CI
GNU nano 7.2 /etc/nginx/sites-available/project1.conf
server {
listen 80;
server_name swostiii.duckdns.org 4.240.110.169;
location / {
proxy_pass http://127.0.0.1:8000;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
}
}
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line

7. Create a Symlink to Enable the Site

Link your config from **sites-available** to **sites-enabled**:

sudo ln -s /etc/nginx/sites-available/project1.conf /etc/nginx/sites-enabled/

```
(venv) swosti@newvm:/etc/nginx/sites-available$ sudo ln -s /etc/nginx/sites-available/project1.conf /etc/nginx/
sites-enabled/
(venv) swosti@newvm:/etc/nginx/sites-available$
```

8. Test NGINX Configuration

Check for syntax errors:

sudo nginx -t

```
(venv) swosti@newvm:/etc/nginx/sites-available$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

If everything is OK, proceed.

9. Reload or Restart NGINX

Apply the changes:

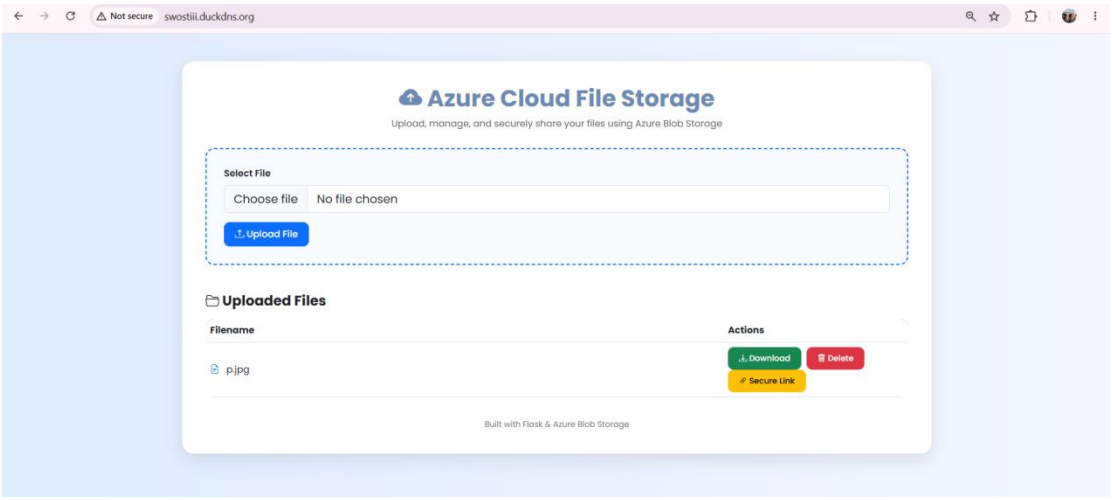
sudo systemctl reload nginx

```
(venv) swosti@newvm:/etc/nginx/sites-available$ sudo systemctl reload nginx
(venv) swosti@newvm:/etc/nginx/sites-available$
```

Or full restart:

sudo systemctl restart nginx

Check the domain you have configure to see apply changes on the domain you configured ***http://swostiii.duckdns.org/***



Lab-5: Virtualization

Objective: To install and configure virtualization software (VirtualBox) on a local machine and create multiple virtual machines.

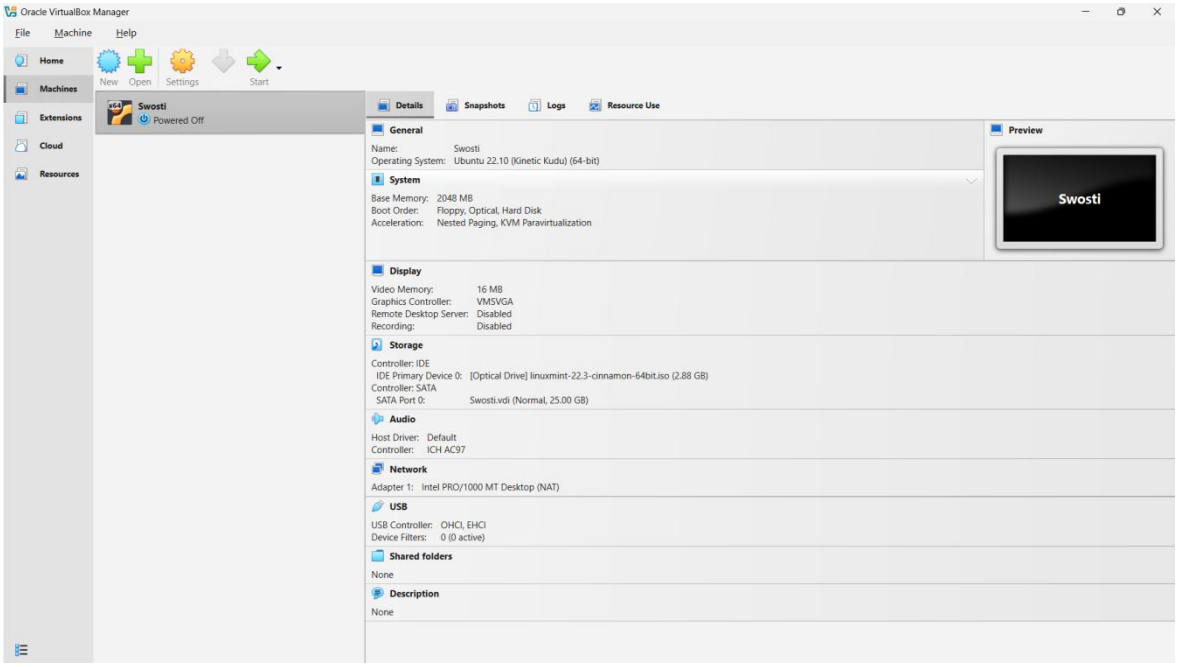
Tool Used:

- Oracle VirtualBox

1. Installing Linux Mint on VirtualBox

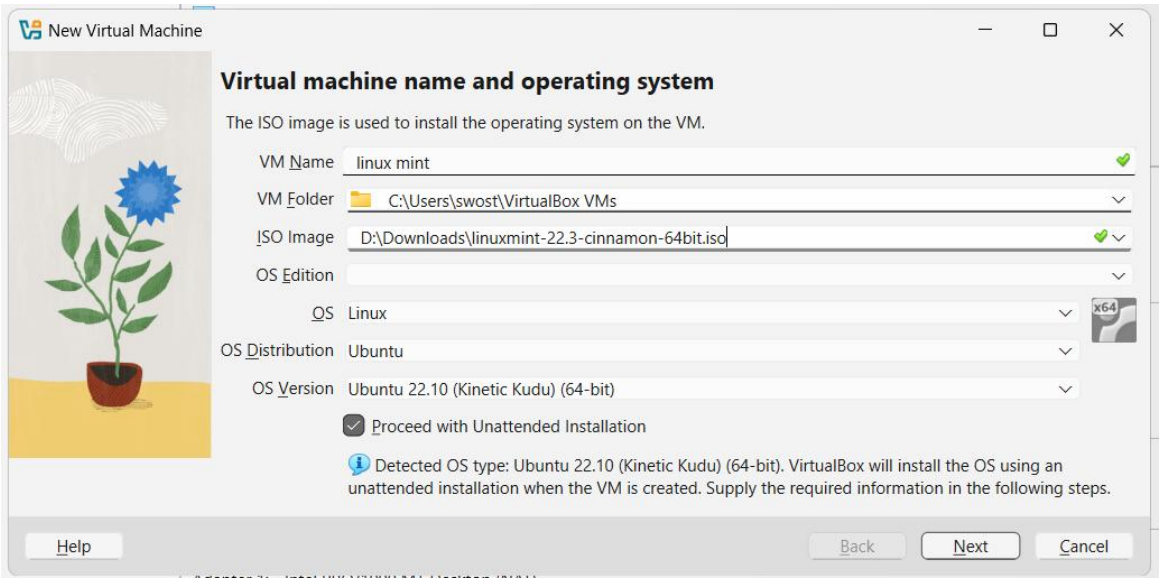
Step-1: Install VirtualBox

1. Download VirtualBox from the official website.
2. Install VirtualBox by following the on-screen instructions.
3. Open VirtualBox after installation.



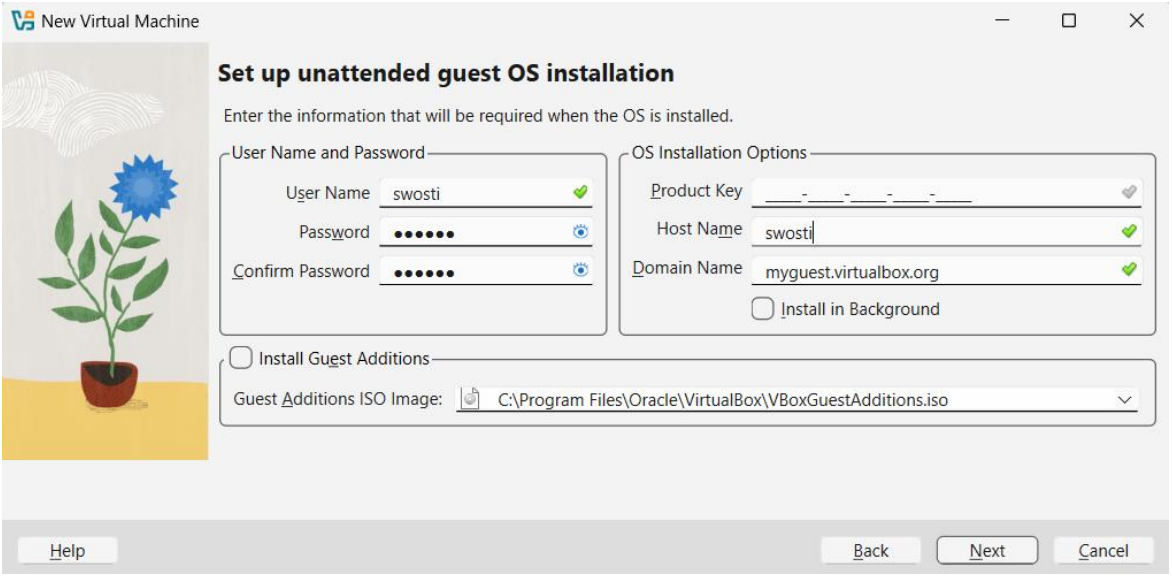
Step-2: Create a New Virtual Machine

1. Click on “New” to create a new virtual machine.
2. A pop-up window will appear. Configure the following:
    a) Name: Enter a name for the VM (e.g. “Linux Mint”).
    b) Folder: Choose a location to store the VM files.
    c) ISO Image: Select the Linux Mint ISO file.



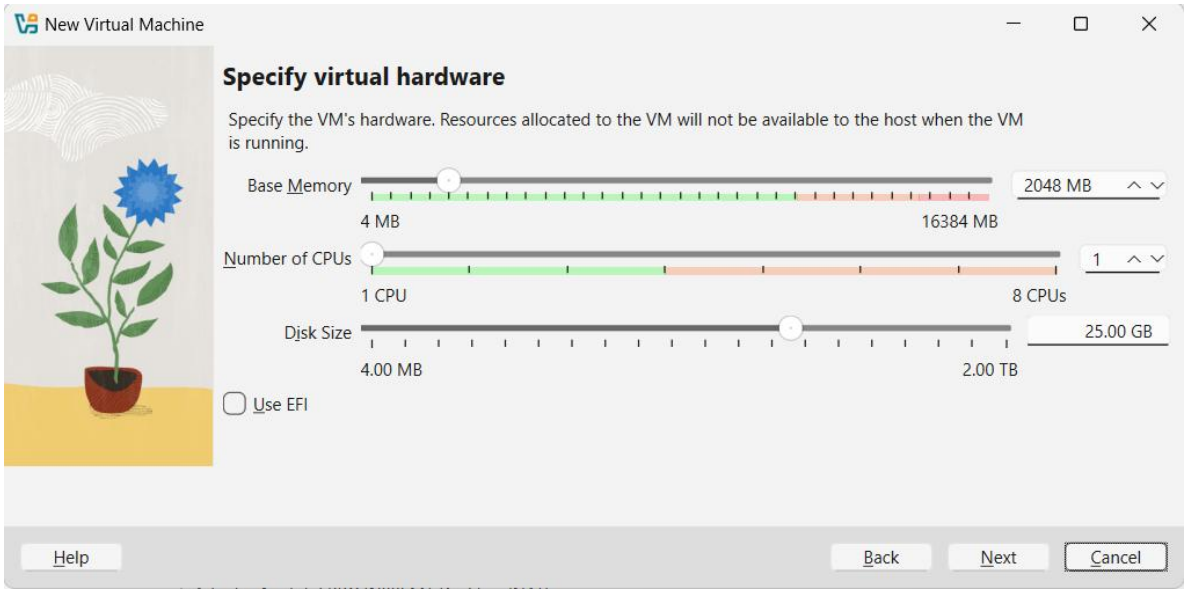
Step-3: Configure System Settings

1. Set up **username, password** and **hostname** for the VM.



Step-4: Customize hardware resources based on system requirements:

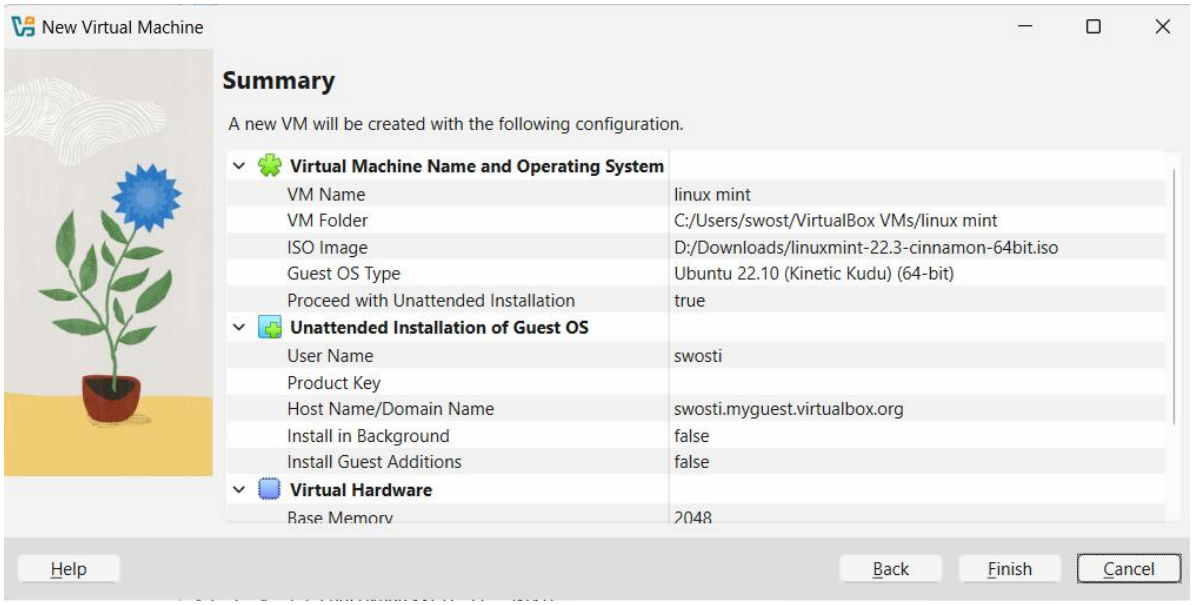
1. **Memory (RAM)** allocation
2. **Processor (CPU) cores**
3. **Storage capacity**



Step-5:Finalizing Setup

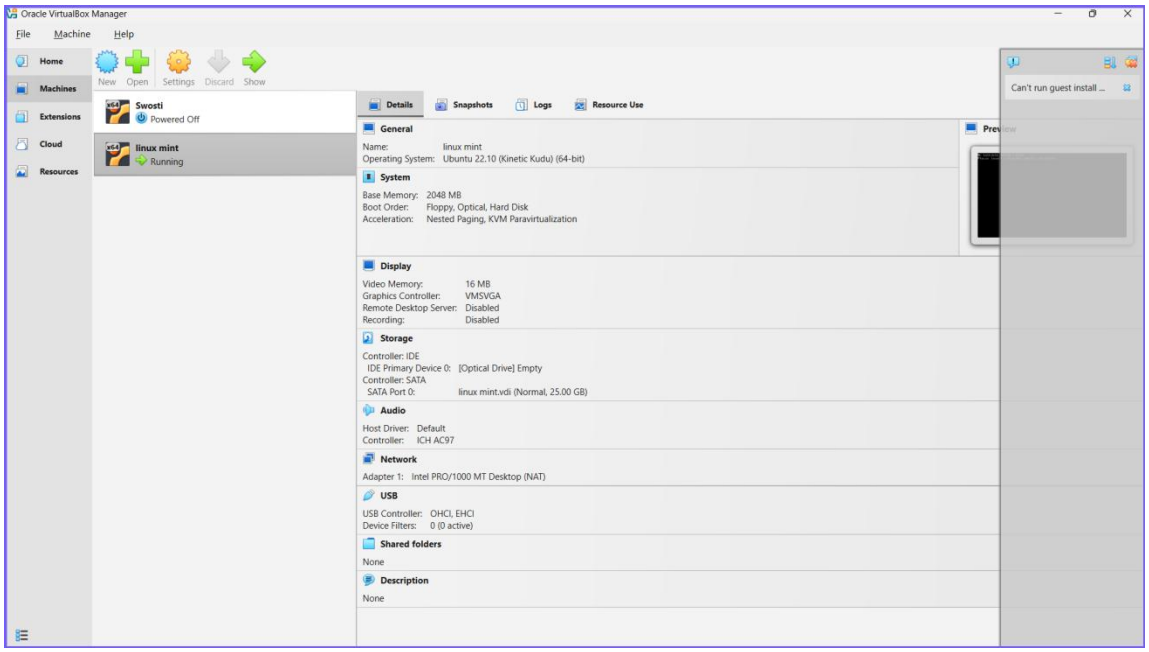
1. Review the summary of the virtual machine configuration.

2. Click "**Finish**" to complete the setup.



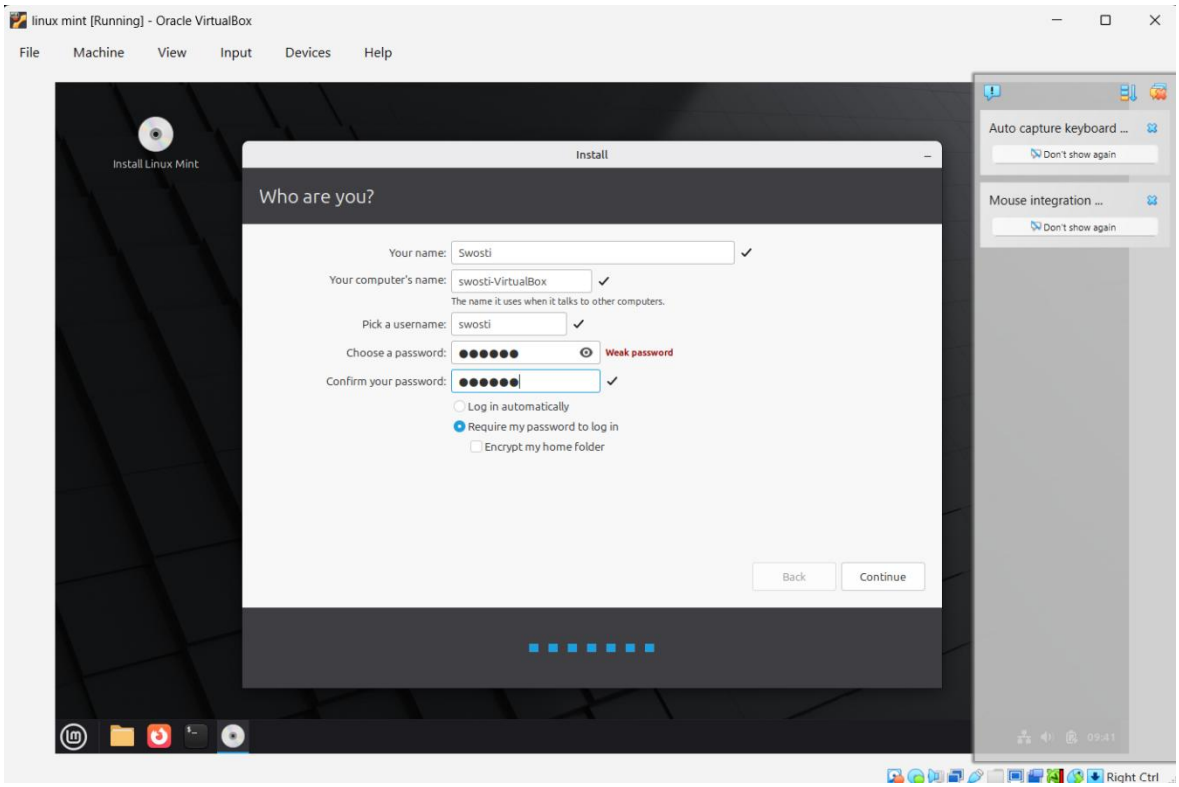
Step-6: Installing Linux Mint

1. Start the virtual machine from the VirtualBox main interface.

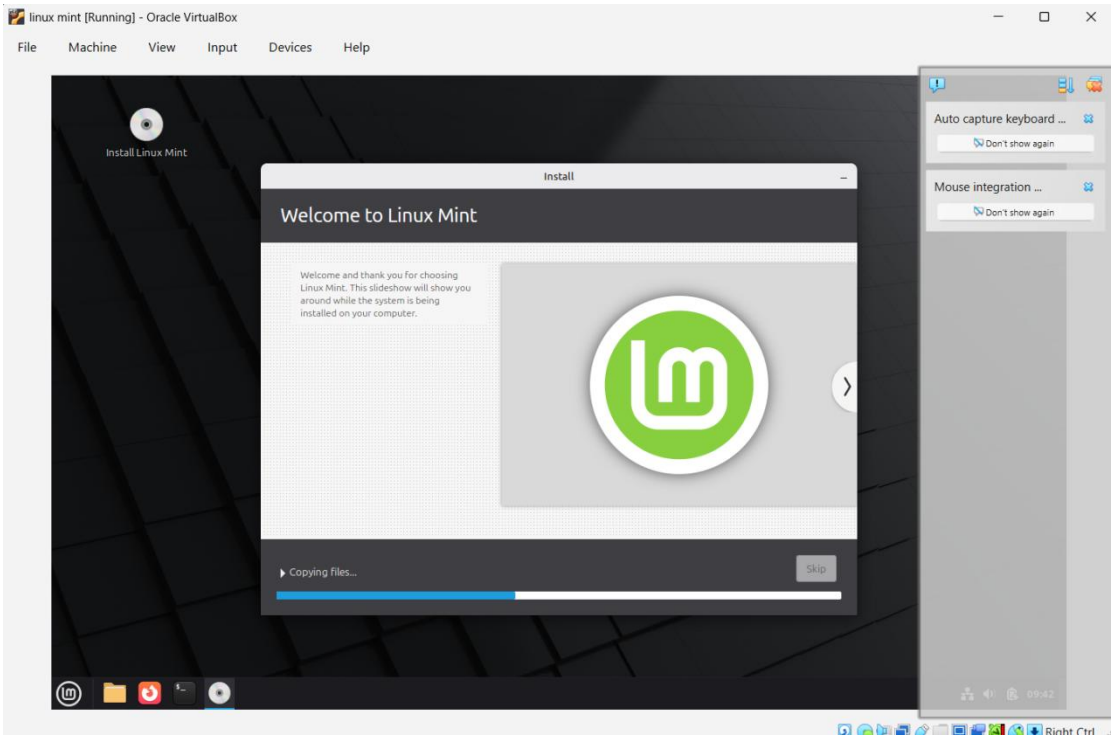


2. The installation process for Linux Mint will begin.
3. Follow on-screen instructions:
   a) Pick a **username** and **password.**

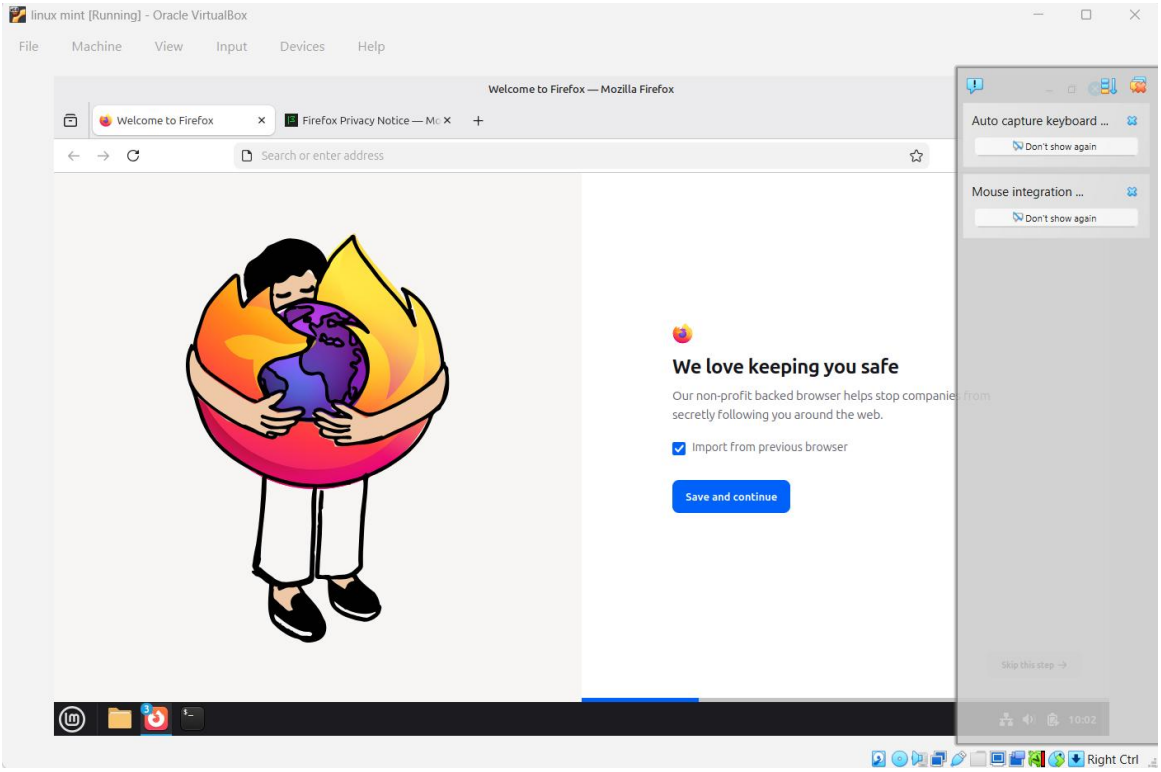
   b) Continue with the installation process.



4. Once completed, **Linux Mint will successfully launch** inside VirtualBox.



5. Successfully launch “Linux Mint” Operating system on Virtual Box.



Lab 6 MapReduce and Parallel Computing

Objective:

Implement and test the MapReduce programming model using Hadoop and Python, perform word count and analyze parallel efficiency and performance improvements.

Tools Used:

- JDK x64(Windows HotSpot)
- Hadoop 3.5.0
- Python 3.12+
- Windows Command Prompt(cmd.exe)

1. Check Java Installation

Installed OpenJDK 8, setup following the instruction and then checked the installed java version using Command Prompt.

a) Java Version

```
C:\Users\acer>java -version
openjdk version "1.8.0_502"
OpenJDK Runtime Environment (Temurin)(build 1.8.0_502-b07)
OpenJDK 64-Bit Server VM (Temurin)(build 25.502-b07, mixed mode)

C:\Users\acer>
```

b) Java Compiler Version

```
C:\Users\acer>javac -version
javac 1.8.0_502
```

2. Extract Hadoop

Download and then extract Apache Hadoop 3.5.0.

Create a new folder on desktop/download (e.g. MapReduce_Lab) and save these two files inside it using Notepad or VS Code:

mapper.py

```
import sys
for line in sys.stdin:
    line = line.strip().lower()
    words = line.split()
```

```
        for word in words:
        print(f"{word}\t1")
```

reducer.py

```
import sys
current_word = None
current_count = 0
for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    try:
        word, count = line.split('\t', 1)
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print(f"{current_word}\t{current_count}")
        current_word = word
        current_count = count
if current_word:
  print(f"{current_word}\t{current_count}")
```

In the same folder, create a text file named input.txt and put some test text in it:

input.txt

```
cat
dog
cat
cow
parrot
dog
```

Then, run these command

a) cd <your folder location>

b) type input.txt | python mapper.py | sort | python reducer.py

```
C:\Users\acer\Downloads\MapReduce_Lab>type input.txt | python mapper.py | so
rt | python reducer.py
cat     2
cow     1
dog     2
parrot  1
```

To evaluate execution performance across varying numbers of parallel processes(N) and dataset sizes (D), speedup and parallel effiency are calculated as follows:

Speedup: $S_N = T_1 / T_N$

Where:

T_1 = execution time with 1 process/node

T_N = execution time with N process/node

Parallel Efficieny: $E_N = S_N / N * 100\%$

Where:

S_N = speedup achieved using N processes/ nodes

N = number of process/node